SIFT: Evaluation on Security Benchmarks

OpenSSF CVE Benchmark | AICGSecEval (Tencent A.S.E)

1. Benchmark 1 — OpenSSF CVE Benchmark (JavaScript)

1.1 Dataset

SIFT was evaluated on the OpenSSF CVE Benchmark, a dataset of 223 real-world JavaScript vulnerabilities spanning 38 CWE categories. Each entry provides a vulnerable version and a patched version of the same file, along with the originating CVE identifier. Two SIFT configurations were compared: a baseline (v1) using standard single-pass code analysis, and an improved version (v2) incorporating diff-aware fix recognition.

1.2 Classification Scheme

Each CVE is classified into one of three outcomes:

Green — SIFT detected the vulnerability AND confirmed the fix is effective.

Orange — SIFT detected the vulnerability but could NOT confirm the fix.

Red — SIFT missed the vulnerability entirely.

1.3 Overall Results (223 CVEs)

System	Green	Orange	Red	F1	Detection Rate
ossf-sast-ml	10%	6%	84%	—	16%
CodeQL	38%	11%	51%	—	49%
SIFT v1 (baseline)	18.4%	37.2%	44.4%	0.311	55.6%
SIFT v2 (diff-aware)	**37.7%**	**22.9%**	**39.5%**	**0.547**	**60.5%**

Table 1. SIFT vs baselines on OpenSSF CVE Benchmark (223 JavaScript CVEs). ossf-sast-ml results from Viszkok & Hegedus (2025) [3]; CodeQL results from OpenSSF benchmark [2].

1.4 SIFT v1 vs v2 — Impact of Diff-Aware Prompting

The sole change between v1 and v2 was the introduction of diff-aware fix recognition: the fixed version is analyzed by providing SIFT with both the vulnerable code, the patched code, and the unified diff, rather than analyzing the patched file in isolation. This change constitutes a clean ablation — only the prompting strategy changed, not the model, threshold, or dataset.

Metric	SIFT v1	SIFT v2	Change	% Improvement
Green rate	18.4%	37.7%	+19.3pp	+105%
Orange rate	37.2%	22.9%	−14.3pp	−38.4%
Precision	0.331	0.622	+0.291	+88%
Recall	0.293	0.488	+0.195	+67%

F1-Score	**0.311**	**0.547**	**+0.236**	**+76%**
Fix recognition rate	33.1%	62.2%	+29.1pp	+88%

Table 2. SIFT v1 vs v2 metric improvements on OpenSSF (n = 223).

A threshold sweep from 40 to 95 confirmed that v2 results are robust to threshold choice (F1 variance < 0.014 across the full range), confirming that performance is determined by the diff-aware prompting strategy rather than score calibration. Notably, SIFT v2's Green rate (37.7%) matches CodeQL and its overall detection rate (60.5%) exceeds ossf-sast-ml by 3.8x. The ossf-sast-ml baseline is a CodeBERT-based neural network analyzer introduced by Viszkok and Hegedus (2025) [3] as part of a unified SAST benchmarking framework on the same OpenSSF dataset — making it the most directly comparable prior AI approach.

1.5 Per-CWE Analysis

Diff-aware prompting produced the largest gains on injection categories where patches consist of targeted, verifiable changes. CWE-088 argument injection (+0.280 F1), CWE-730 ReDoS (+0.286 F1, the largest relative gain at +152%), and CWE-022 path traversal (+0.223 F1) benefited most. CWE-770 allocation without limits scored F1 = 0.000 in both versions, reflecting the inherent difficulty of reasoning about computational complexity bounds without formal analysis.

CWE Category	F1 (v1)	F1 (v2)	Delta	n
CWE-088 Argument Injection	0.438	0.718	+0.280	19
CWE-730 ReDoS	0.188	0.474	+0.286	29
CWE-022 Path Traversal	0.444	0.667	+0.223	23
CWE-078 OS Command Injection	0.400	0.609	+0.209	31
CWE-770 Alloc. Without Limits	0.000	0.000	0.000	8

Table 3. Per-CWE F1 comparison: SIFT v1 vs v2 (top categories by frequency).

1.6 Key Finding

Fix recognition is a prompt design problem, not a fundamental LLM limitation. SIFT v1's 37.2% Orange rate was not caused by an inability to reason about security, but by asking the model to classify a patched file without providing the original vulnerable context. Supplying the diff converts the task from open-ended classification to targeted patch verification — a fundamentally easier problem for an LLM. SIFT v2's Green rate (37.7%) matches CodeQL while detection rate (60.5%) exceeds the ossf-sast-ml baseline [3] by 3.8x, despite ossf-sast-ml being specifically trained for JavaScript vulnerability detection using CodeBERT embeddings.

---

2. Benchmark 2 — AICGSecEval (Tencent A.S.E, Multi-Language)

2.1 Dataset

SIFT was additionally evaluated on the AICGSecEval benchmark (Lian et al., 2025), a repository-level security evaluation dataset covering 129 instances across 7 programming

languages (C, C++, Java, JavaScript, PHP, Python, TypeScript) and 29 CWE categories. The dataset is derived from real-world GitHub repositories with documented CVEs. Unlike the OpenSSF benchmark, all code was fetched directly from GitHub at the vulnerable and patched commits — no local files were available.

Note on task framing: The original A.S.E paper evaluates LLMs on code generation security (producing secure code from scratch). SIFT's task is complementary — static vulnerability detection and patch verification on the same CVE instances. The dataset is repurposed here for detection evaluation; the two tasks are not directly comparable in results, but the benchmark provides a challenging multi-language testbed.

2.2 Engineering Challenges and Solutions

Two prompt engineering improvements were developed specifically for this benchmark:

Fix 1 — Vulnerability-centered truncation: The OpenSSF benchmark's head+tail truncation strategy was inadequate here because C source files frequently exceed 20,000 characters. Standard truncation placed the vulnerable lines in the discarded middle section, causing SIFT to report 'only comment changes visible' and produce false Orange outcomes. The fix anchors the context window around the known vuln_lines range (provided in the dataset), preserving 80 lines of context before and after the vulnerable region.

Fix 2 — Focused diff injection: For fix recognition, the unified diff between vulnerable and patched versions (computed from full files, not truncated copies) is prepended to the analysis prompt. This guarantees the exact patch is always visible to the model regardless of file size, enabling accurate fix verification even on large C codebases.

2.3 Results — Ablation Across Three Configurations

Configuration	Green	Orange	Precision	F1
Baseline (head+tail truncation)	27.1%	51.9%	0.343	0.427
+ Fix 1: Vulnerability-centered truncation	48.1%	34.1%	0.585	0.649
+ Fix 2: Focused diff injection	**73.6%**	**11.6%**	**0.864**	**0.848**

Table 4. AICGSecEval results across three SIFT configurations (n = 129).

Fix 1 alone (vulnerability-centered truncation) raised F1 from 0.427 to 0.649 (+52%), primarily by resolving the C-language truncation problem. Fix 2 (focused diff injection) further raised F1 to 0.848, reducing the Orange rate from 34.1% to 11.6% by ensuring the model always sees the actual patch.

2.4 Per-Language Breakdown (Final Configuration)

Language	n	Green%	Orange%	Red%	F1
Java	16	93.8%	0.0%	6.2%	**0.968**
C	54	85.2%	7.4%	7.4%	**0.920**
PHP	42	64.3%	19.0%	16.7%	0.783
Python	13	53.8%	23.1%	23.1%	0.700

Table 5. Per-language results, SIFT with Fix 1+2 on AICGSecEval (n = 129).

Java achieved the highest F1 (0.968), followed by C (0.920) — a language traditionally considered difficult for LLM-based analysis. PHP (0.783) and Python (0.700) also performed strongly. The C result is particularly notable: at baseline, C scored F1 = 0.258 due to truncation. After Fix 1+2, it reached 0.920 — a 256% improvement attributable entirely to context delivery, not model capability.

2.5 Key Finding

LLM-based vulnerability analysis on systems-language code (C, C++) is not fundamentally limited by reasoning capability — it is a context delivery problem. When the model is shown the relevant vulnerable lines and the exact diff, it accurately identifies and verifies fixes even in complex C codebases. This finding generalises the OpenSSF result: diff-aware, context-centered prompting is effective across languages, not just JavaScript.

---

3. Cross-Benchmark Summary

System	Dataset	Languages	Green%	Recall	F1
ossf-sast-ml	OpenSSF	JS	10%	—	—
CodeQL	OpenSSF	JS	38%	—	—
SIFT v1	OpenSSF	JS	18.4%	0.293	0.311
SIFT v2	OpenSSF	JS	37.7%	0.488	0.547
SIFT (Fix 1+2)	**AICGSecEval**	**7 languages**	**73.6%**	**0.833**	**0.848**

Table 6. Complete results across both benchmarks and all configurations.

Across both benchmarks, SIFT demonstrates that vulnerability detection and fix recognition are fundamentally prompt engineering problems rather than model capability limitations. The consistent gains from diff-aware prompting (OpenSSF: +76% F1) and vulnerability-centered context selection (AICGSecEval: +52% F1 from Fix 1 alone) confirm that supplying the model with precisely the right context — the vulnerable code, the patch, and the diff — is the dominant factor in detection quality. The final configuration (Fix 1+2) on AICGSecEval achieves F1 = 0.848 across 7 languages, substantially outperforming SIFT v2 on the JavaScript-only OpenSSF benchmark (F1 = 0.547), and suggests that the approach generalizes effectively to multi-language, repository-level security analysis.

References

[1] Lian, K., Wang, B., Zhang, L., et al. (2025). A.S.E: A Repository-Level Benchmark for Evaluating Security in AI-Generated Code. arXiv:2508.18106.

[2] OpenSSF CVE Benchmark. https://github.com/ossf/sast-scan-cve-benchmark

[3] Viszkok, T., & Hegedus, P. (2025). Unified SAST benchmark: Compare AI-driven and traditional static analyzers. SoftwareX, 31, 102336. https://doi.org/10.1016/j.softx.2025.102336